



Pontificia Universidad Javeriana
Facultad de Ingenieria
Fundamentos de Ingenieria de Software

Informe de Postmortem

Reporte del ciclo, retrospectiva Starfish y PIP

Milestone 3: Gestión Documental, Monitoreo y Nube

Equipo SYNTIX TECH

Profesor	Luis Gabriel Moreno Sandoval
Monitor	Juan Pablo Arias Buitrago
Fecha	19 de mayo de 2026
Repositorio	https://github.com/puj-course/FIS_2610_3517_G4

Documento construido a partir del postmortem original del milestone, sin recortar informacion, adaptado al formato institucional de la plantilla suministrada.

Integrantes

Nombre	GitHub	Rol Scrum / tecnico
Sebastian Ramirez Maldonado	@Sarm-m	Scrum Master
Sebastian Vargas	@juanvargax	Product Owner / Sprint Planner
Samuel Freile	@samuelfl680	Configuration Manager
Solon Losada	@solonlosada2006	DevOps Engineer
Sebastian Rodriguez Ramirez	@juserora	QA Lead

Tabla 1: Integrantes y roles del equipo SYNTIX TECH.

Tabla de contenido

Integrantes	1
1. Introduccion	4
2. Postmortem – Milestone 3: Gestión Documental, Monitoreo y Nube	5
2.1. 1. Resumen Ejecutivo	5
2.2. 2. Contexto de Ingeniería y Negocio	5
2.3. 3. Revisión de Datos del Proceso y Calidad	5
2.3.1. Desempeño real vs. planeado	6
2.3.2. Lecciones aprendidas	6
2.4. 4. Reporte de Roles	6
2.4.1. 4.1 Reporte del Scrum Master – Sebastián Ramírez Maldonado (@Sarm-m)	6
2.4.2. 4.2 Reporte del Product Owner / Sprint Planner – Sebastián Vargas (@juanvargax)	7
2.4.3. 4.3 Reporte del Configuration Manager – Samuel Freile (@samuelff680)	7
2.4.4. 4.4 Reporte del Quality Assurance Lead – Sebastián Rodríguez Ramírez (@Juserora)	7
2.4.5. 4.5 Reporte del DevOps Engineer – Solón Losada (@solonlosada2006) .	7
2.5. 5. Identificación de Causa Raíz	8
2.6. 6. Retrospectiva	8
2.7. 7. Retrospectiva Starfish	8
2.7.1. Seguir haciendo Seguir Haciendo	8
2.7.2. Comenzar a hacer Comenzar a Hacer	9
2.7.3. Dejar de hacer Dejar de Hacer	9
2.7.4. Mas de Más de	10
2.7.5. Menos de Menos de	10
2.8. 8. Plan de Mejora para el Milestone 4	10
2.9. 9. Respuestas completas a las preguntas guía de la PPT de Postmortem	11
2.9.1. 9.1 Revisión del producto, esfuerzo y proceso	11
2.9.2. 9.2 Evaluación objetiva de roles	12
2.9.3. 9.3 Reporte del ciclo por responsabilidades	14
2.9.4. 9.4 Estrategia de desarrollo y atributos de calidad	14

2.9.5. 9.5 Administración de configuración, riesgos y reutilización 15

2.9.6. 9.6 Reflexión del trabajo en grupo 15

2.9.7. 9.7 Verificación Starfish: respuestas por eje 15

2.9.8. 9.8 PIP - Plan de Mejora del Proceso 16

1. Introduccion

Este informe presenta el postmortem del Milestone 3 del proyecto DriveControl / AutoMinder Enterprise, desarrollado por el equipo SYNTIX TECH en el marco de la asignatura Fundamentos de Ingenieria de Software. El documento conserva de forma integral la informacion del postmortem original y la reorganiza bajo el formato institucional suministrado, manteniendo el contenido de resumen ejecutivo, contexto, revision de calidad, evaluacion de roles, causa raiz, retrospectiva Starfish, plan de mejora y respuestas completas a las preguntas guia de la presentacion de postmortem.

El proposito de este reporte es dejar una evidencia clara del ciclo: que producto se produjo, que esfuerzo se invirtio, que proceso se siguio, que problemas aparecieron, cuales fueron sus causas, que aprendizajes surgieron y que acciones de mejora deben incorporarse en el ciclo siguiente o en proyectos futuros. La estructura facilita la lectura academica y permite defender el trabajo con trazabilidad metodologica, tecnica y de calidad.

2. Postmortem – Milestone 3: Gestión Documental, Monitoreo y Nube

Proyecto: Drive Control / Syntix Tech **Stack:** MERN (MongoDB, Express, React, Node.js)
Equipo:

Nombre	Usuario	Rol
Sebastián Ramírez Maldonado	@Sarm-m	Scrum Master
Samuel Freile	@samuelff680	Configuration Manager
Sebastián Rodríguez Ramírez	@Juserora	Quality Assurance Lead
Solón Losada	@solonlosada2006	DevOps Engineer
Sebastián Vargas	@juanvargax	Product Owner / Sprint Planner

2.1. 1. Resumen Ejecutivo

El Milestone 3 representó el salto más significativo en la madurez técnica del proyecto: la migración del almacenamiento local hacia una infraestructura de nube real mediante MongoDB Atlas, la implementación de aislamiento de datos por usuario mediante validaciones de `ownerEmail`, y la formalización del proceso de QA con reportes estandarizados. La incorporación de Scrum Poker mejoró notablemente la asertividad de las estimaciones. Sin embargo, el hito también expuso una deuda técnica de seguridad crítica: la postergación de la configuración de variables de entorno (`.env`) para las fases finales del sprint resultó en credenciales temporalmente expuestas en el repositorio, lo que fue detectado por el pipeline de SonarCloud y generó retrasos en el flujo de integración.

2.2. 2. Contexto de Ingeniería y Negocio

El valor de negocio central del Milestone 3 fue garantizar la persistencia, disponibilidad y privacidad de los datos de la flota en un entorno real de producción. MongoDB Atlas proporcionó la infraestructura de nube necesaria para que los datos de los vehículos y conductores sobrevivieran a reinicios del servidor y pudieran ser accedidos desde cualquier entorno. La validación por `ownerEmail` aseguró el aislamiento de datos entre usuarios, un requisito de privacidad fundamental para una plataforma multiusuario.

Desde la perspectiva de ingeniería, la estabilización de la edición de vehículos y conductores y la implementación de la asignación entre entidades relacionales (Conductor-Vehículo) completó el modelo de dominio central del sistema, sobre el cual los módulos de alertas y monitoreo del M4 podrían construirse con bases sólidas.

2.3. 3. Revisión de Datos del Proceso y Calidad

2.3.1. Desempeño real vs. planeado

- La adopción de Scrum Poker permitió estimaciones más cercanas a la realidad, reduciendo el gap entre el esfuerzo planeado y el ejecutado respecto a los milestones anteriores.
- El rol de QA fue formalizado exitosamente: los hallazgos se reportaron con pasos de reproducción claros y fueron registrados en GitHub Issues con un template estandarizado antes del cierre de cada historia de usuario.
- El pipeline de CI/CD detectó credenciales e hilos de conexión hardcodeados en el repositorio, lo que bloqueó temporalmente el flujo de integración y requirió una sesión de remediación no planificada.
- El QA detectó discrepancias visuales en el componente StatusBadge bajo pantallas móviles, que no habían sido consideradas en los criterios de aceptación originales.
- La validación de relaciones complejas (Conductor-Vehículo) generó más esfuerzo del estimado por la necesidad de implementar validaciones en múltiples capas (modelo, ruta, middleware).

2.3.2. Lecciones aprendidas

- Los archivos `.env` deben configurarse en el Día 1 del sprint, no como una tarea final. Su postergación es un riesgo de seguridad activo, no un detalle administrativo.
 - SonarCloud como herramienta de análisis estático es un activo de calidad, pero también un bloqueador si se activa tarde: integrarla desde el inicio del proyecto es siempre más económico que remediar sus hallazgos bajo presión de entrega.
 - La responsividad del componente StatusBadge no fue cubierta en los criterios de aceptación, evidenciando que las historias de usuario deben incluir criterios de responsive design explícitos cuando involucran componentes de visualización de estado.
-

2.4. 4. Reporte de Roles

2.4.1. 4.1 Reporte del Scrum Master – Sebastián Ramírez Maldonado (@Sarm-m)

El Scrum Master lideró la adopción de Scrum Poker en este milestone, lo que tuvo un efecto positivo directo en la calidad de las estimaciones y en el sentido de propiedad colectiva sobre los compromisos del sprint. La visibilidad del proceso mejoró respecto a los milestones anteriores, aunque la detección tardía de las credenciales hardcodeadas reveló que aún falta un espacio explícito en el sprint para revisar riesgos técnicos de seguridad antes de integrar cambios. Para el M4, el Scrum Master debe incorporar una revisión de riesgos de seguridad como ítem fijo en la agenda del Sprint Planning.

2.4.2. 4.2 Reporte del Product Owner / Sprint Planner – Sebastián Vargas (@juanvargax)

El Product Owner logró mantener la coherencia entre las alertas documentales y las entidades del modelo de dominio, aunque la validación de relaciones complejas entre Conductores y Vehículos demandó más esfuerzo del estimado. Las historias de usuario del M3 no incluyeron criterios de aceptación explícitos para la responsividad de los componentes de visualización, lo que permitió que el hallazgo del StatusBadge en móviles escapara al primer ciclo de revisión. Para el M4, todos los criterios de aceptación de historias que involucren componentes visuales deben incluir un escenario de prueba en dispositivo móvil.

2.4.3. 4.3 Reporte del Configuration Manager – Samuel Freile (@samuelf680)

El Configuration Manager identificó el problema crítico de las credenciales hardcodeadas, pero la detección ocurrió a través del pipeline de SonarCloud en lugar de en una revisión de código previa. Esto evidencia que la política de gestión de variables de entorno no estaba suficientemente formalizada en el equipo. Para el M4, el Configuration Manager debe implementar un template de `.env.example` en el repositorio desde el Día 1 del sprint, y añadir una verificación automatizada en el pipeline que detecte patrones de credenciales hardcodeadas antes de que el código llegue a SonarCloud.

2.4.4. 4.4 Reporte del Quality Assurance Lead – Sebastián Rodríguez Ramírez (@Juserora)

El Milestone 3 fue el hito en el que el rol de QA alcanzó su mayor madurez hasta el momento. Los reportes de hallazgos fueron estandarizados con pasos de reproducción claros, lo que redujo significativamente el tiempo de triaje y corrección. El hallazgo del StatusBadge en pantallas móviles fue el caso más representativo: fue documentado con capturas de pantalla, pasos de reproducción y el dispositivo/resolución afectada, permitiendo una corrección precisa. Para el M4, el QA Lead debe incorporar pruebas en dispositivos móviles como criterio de cierre obligatorio para cualquier historia de usuario que incluya componentes de visualización de estado o datos.

2.4.5. 4.5 Reporte del DevOps Engineer – Solón Losada (@solonlosada2006)

El DevOps Engineer tuvo su milestone más crítico hasta el momento: el pipeline de CI/CD fue el mecanismo que detectó las credenciales hardcodeadas, validando la inversión en infraestructura de integración continua. Sin embargo, la detección en SonarCloud también evidenció que la política de gestión de secretos no había sido comunicada con suficiente énfasis al equipo antes de comenzar el sprint. Para el M4, el DevOps Engineer debe configurar pre-commit hooks locales que detecten patrones de credenciales hardcodeadas antes de que el código llegue al repositorio remoto, actuando como primera línea de defensa antes de SonarCloud.

2.5. 5. Identificación de Causa Raíz

La causa raíz principal del Milestone 3 fue la **postergación de la configuración de variables de entorno de producción (.env) para las fases finales del sprint**, en lugar de implementarlas desde el Día 1. Esta decisión, tomada implícitamente por ausencia de una política formal, resultó en credenciales e hilos de conexión temporalmente expuestos en el repositorio, lo que fue detectado por SonarCloud como vulnerabilidades críticas y bloqueó el pipeline de integración en un momento de alta presión de entrega.

Como causa secundaria, la **ausencia de criterios de aceptación de responsividad** en las historias de usuario que involucran componentes visuales permitió que el hallazgo del StatusBadge en pantallas móviles superara el primer ciclo de revisión, generando una corrección tardía que consumió tiempo del sprint.

2.6. 6. Retrospectiva

El Milestone 3 fue el sprint en que el equipo sintió por primera vez el peso real de la deuda técnica de seguridad. No se trató de un ataque externo ni de una vulnerabilidad compleja: fue simplemente no haber configurado el `.env` desde el principio. Es una lección que parece obvia en retrospectiva, pero que en la presión del sprint se postergó como una tarea "menor" para el final. El pipeline de SonarCloud no mintió: las credenciales estaban ahí, visibles para cualquiera que tuviera acceso al repositorio.

Sin embargo, el M3 también fue el sprint donde el equipo demostró su mayor madurez como proceso: Scrum Poker funcionó, el QA fue riguroso, los reportes de bugs fueron precisos y accionables, y la migración a MongoDB Atlas se completó sin interrupciones de servicio. El equipo aprendió que la calidad no es un atributo que se añade al final, sino una disciplina que se practica en cada decisión del sprint, incluyendo las más pequeñas, como cuándo crear el archivo `.env`.

La formalización del rol de QA fue el cambio estructural más valioso del milestone. Tener un proceso claro de reporte de hallazgos, con pasos de reproducción y criterios de aceptación verificables, transformó la calidad de una actividad informal en una responsabilidad medible. Esta disciplina debe mantenerse y profundizarse en el M4.

El equipo cierra el M3 más consciente de que la seguridad no es una feature: es un requisito transversal que debe estar presente desde el Día 1 de cada sprint.

2.7. 7. Retrospectiva Starfish

***Reglas aplicadas:** cada frase inicia con un verbo en infinitivo; ninguna frase se repite entre ejes.*

2.7.1. Seguir haciendo Seguir Haciendo

#	Frase	Justificación
1	Refactorizar los modelos de datos antes de modificar las rutas y controladores dependientes.	Garantiza que el contrato de datos sea estable antes de que otras capas del sistema dependan de él.
2	Aplicar Peer Reviews rigurosos en cada Pull Request, verificando lógica de negocio y no solo compilación.	Los PR del M3 con mayor profundidad de revisión fueron los que introdujeron menos bugs en integración.
3	Realizar pruebas del sistema en diversos dispositivos y resoluciones de pantalla.	El hallazgo del StatusBadge en móviles hubiera llegado más tarde al equipo sin esta práctica.
4	Usar MongoDB Atlas como infraestructura de nube para garantizar la persistencia real de los datos.	Eliminó la dependencia de bases de datos locales y habilitó el acceso multi-entorno al sistema.
5	Mantener la trazabilidad de cada historia de usuario mediante sub-issues técnicas en GitHub.	Permite reconstruir el histórico de decisiones técnicas de cada funcionalidad para el postmortem y para el mantenimiento futuro.

2.7.2. Comenzar a hacer Comenzar a Hacer

#	Frase	Justificación
1	Configurar el archivo <code>.env</code> y las variables de entorno desde el Día 1 de cada sprint, sin excepciones.	La postergación de esta configuración fue la causa raíz del incidente de seguridad del M3.
2	Implementar middlewares globales de autenticación y validación desde el inicio del sprint, antes de construir rutas específicas.	Los middlewares globales son la capa de seguridad más eficiente: aplican a todas las rutas sin repetición de código.
3	Automatizar la limpieza de bases de datos de prueba entre sesiones de QA para garantizar datos consistentes.	Los datos de prueba residuales de sesiones anteriores pueden generar falsos positivos o negativos en las verificaciones de QA.
4	Incluir criterios de aceptación de responsive design en todas las historias de usuario con componentes visuales.	El hallazgo del StatusBadge en móviles evidenció que la responsividad no puede ser un criterio implícito.
5	Usar un template estandarizado de reportes de bugs con pasos de reproducción, entorno y evidencia fotográfica.	Reduce el tiempo de triaje y permite a los desarrolladores reproducir y corregir el hallazgo sin back-and-forth con el QA.
6	Configurar pre-commit hooks locales que detecten credenciales hardcodeadas antes de que lleguen al repositorio remoto.	SonarCloud es la última línea de defensa; los hooks locales son la primera y la más eficiente.

2.7.3. Dejar de hacer Dejar de Hacer

#	Frase	Justificación
1	Incluir credenciales o cadenas de conexión hardcodeadas en el código fuente del repositorio.	Es una vulnerabilidad de seguridad crítica que expone datos sensibles a cualquier persona con acceso al repositorio.
2	Depender de datos simulados o estáticos para validar flujos que requieren datos reales de la base de datos.	Los datos simulados no cubren los casos de borde que emergen con datos reales de producción.
3	Posponer los ajustes de configuración de entorno para el staging hasta las fases finales del sprint.	La configuración tardía del entorno comprime el tiempo disponible para las pruebas de integración y QA.

#	Frase	Justificación
4	Ignorar las discrepancias visuales del componente StatusBadge bajo distintas resoluciones de pantalla.	Un componente de estado visible en todas las pantallas del sistema debe ser consistente en todos los dispositivos.
5	Desarrollar flujos completos sin documentar los casos de prueba asociados antes de iniciar el QA.	Sin casos de prueba documentados, el QA opera de forma reactiva en lugar de estructurada.

2.7.4. Mas de Más de

#	Frase	Justificación
1	Aplicar Scrum Poker para estimar el esfuerzo de todas las historias de usuario del sprint.	En el M3 demostró mejorar la asertividad de los compromisos y el sentido de propiedad colectiva sobre el sprint.
2	Mantener coherencia entre las alertas documentales y las entidades del modelo de dominio en tiempo real.	Una alerta que no corresponde a la entidad correcta del modelo es un error de negocio, no solo un bug visual.
3	Comunicar de forma asertiva cualquier bloqueo técnico al equipo en cuanto se identifica, sin esperar a la Daily.	Los bloqueos comunicados tardíamente se convierten en impedimentos que consumen tiempo de varios integrantes.
4	Validar tempranamente los flujos críticos del sistema antes de construir las capas de presentación sobre ellos.	Es más económico corregir una ruta de API mal definida antes de que el frontend dependa de ella.
5	Usar herramientas de desarrollo responsivo (DevTools, emuladores de dispositivos) como parte del ciclo de desarrollo normal.	La responsividad verificada durante el desarrollo es más barata que la corregida durante el QA.

2.7.5. Menos de Menos de

#	Frase	Justificación
1	Generar inconsistencias en los resultados de los buscadores por datos de prueba residuales entre sesiones.	Los datos residuales contaminan el entorno de prueba y generan resultados no reproducibles.
2	Introducir fallos de diseño en tablas responsivas que no fueron verificadas en dispositivos móviles.	Las tablas son el componente más susceptible a problemas de responsividad en sistemas de gestión de flota.
3	Generar retrasos en el pipeline por credenciales no gestionadas que debieron configurarse antes de iniciar el desarrollo.	Cada retraso en el pipeline en la fase final del sprint es tiempo que se roba directamente al QA y a la entrega.
4	Perder el alcance del sprint por no ajustar el backlog cuando emergen tareas no planificadas de seguridad.	Las tareas de remediación de seguridad deben entrar al sprint con una historia de usuario formal, desplazando trabajo de menor prioridad.
5	Redactar reportes de hallazgos vagos sin pasos de reproducción ni evidencia que permitan la corrección precisa.	Un reporte vago obliga al desarrollador a reproducir el contexto desde cero, duplicando el esfuerzo total de resolución.

2.8. 8. Plan de Mejora para el Milestone 4

Acción	Responsable	Criterio de éxito
Configurar <code>.env</code> y variables de entorno en el Día 1 del M4 como primera tarea del sprint.	Configuration Manager (@samuel680)	Commit de configuración de <code>.env.example</code> en el primer día del M4.
Implementar pre-commit hooks para detección de credenciales hardcodeadas.	DevOps Engineer (@solonlosada2006)	El hook rechaza automáticamente commits con patrones de credenciales.
Incluir criterios de aceptación de responsividad en todas las historias del M4 con componentes visuales.	Product Owner (@juanvargax)	Cada historia con componente visual tiene al menos un criterio de aceptación en móvil.
Automatizar la limpieza de la base de datos de prueba entre ciclos de QA.	QA Lead (@Juserora)	Script de limpieza ejecutado y documentado antes de cada sesión de QA del M4.
Incorporar revisión de riesgos de seguridad en la agenda del Sprint Planning del M4.	Scrum Master (@Sarm-m)	Checklist de riesgos de seguridad completa en el Sprint Planning del M4.

2.9. 9. Respuestas completas a las preguntas guía de la PPT de Postmortem

Esta sección se agrega como matriz de verificación para dejar respondidas de forma explícita todas las preguntas guía de la presentación de Postmortem y Retrospectiva Starfish. Las respuestas se formulan con hechos del milestone, evitando evaluar personas y concentrándose en el desempeño del trabajo, el producto, el proceso y las oportunidades de mejora.

2.9.1. 9.1 Revisión del producto, esfuerzo y proceso

Pregunta guía de la PPT	Respuesta aplicada al Milestone 3: Gestión Documental, Monitoreo y Nube
¿Qué producto se produjo?	Se produjo la gestión documental conectada a MongoDB Atlas, aislamiento de datos por <code>ownerEmail</code> , estabilización de edición de vehículos/conductores, asignación Conductor-Vehículo y formalización del QA con reportes reproducibles.
¿Qué esfuerzo se invirtió para hacerlo?	El esfuerzo se destinó a migración a nube, validaciones de privacidad, relaciones de dominio y correcciones derivadas del análisis de seguridad y responsividad.
¿Qué proceso se siguió para hacerlo?	El proceso maduró con Scrum Poker, QA formal y CI/CD, pero falló al dejar la configuración de variables de entorno para fases tardías.
¿Cómo fue el desempeño real comparado con lo planeado?	El desempeño fue alto frente a lo planeado en persistencia y QA; sin embargo, hubo retrasos por credenciales hardcodeadas detectadas por SonarCloud y por correcciones visuales no previstas.
¿Qué lecciones se aprendieron de esta experiencia?	La seguridad y el entorno no son tareas finales: se configuran desde el Día 1. Además, los criterios de aceptación visual deben incluir dispositivos móviles.

Pregunta guía de la PPT	Respuesta aplicada al Milestone 3: Gestión Documental, Monitoreo y Nube
¿Debería usarse un criterio diferente, a nivel individual o de equipo, para el futuro?	Sí. Se debe usar un criterio de cierre que incluya revisión de secretos, validación de <code>ownerEmail</code> , pruebas móviles y reporte QA con pasos de reproducción.
¿Dónde hay oportunidades para mejorar y por qué?	La mayor oportunidad es transformar la seguridad y QA en controles preventivos: <code>.env.example</code> , hooks locales, criterios responsive y limpieza de datos de prueba.
¿Dónde hubo problemas que deben corregirse para el próximo ciclo?	Se deben corregir la postergación de variables de entorno, la falta de criterios móviles y la validación tardía de relaciones complejas.
¿Qué hizo bien el equipo y en qué falló?	Hizo bien: usar Scrum Poker, formalizar QA, migrar a MongoDB Atlas y mejorar persistencia real. Falló en: configurar tarde secretos, dejar responsividad implícita y subestimar validaciones Conductor-Vehículo
¿Sirvieron las mejoras propuestas previamente?	Sí. Las mejoras de trazabilidad y reportes sirvieron: las issues de QA fueron más claras y Scrum Poker redujo estimaciones ingenuas. Faltó aplicar con igual rigor la gestión de secretos.
¿Cómo ha sido el desempeño comparado con otros equipos?	No se cuenta con datos verificables de otros equipos, por lo que no se afirma una comparación externa. La comparación válida es interna: frente a M1 y M2, el M3 fue más maduro en QA, estimación y persistencia; su principal brecha fue seguridad preventiva.
¿Se deben modificar los objetivos y las metas?	Sí. Las metas del M4 deben incluir seguridad, cobertura, pruebas E2E, despliegue y limpieza de datos, no solo features.
¿Cómo se deben modificar los procesos?	Incluir revisión de riesgos de seguridad en Sprint Planning, hooks locales para secretos y criterios responsive por historia visual.
¿Cuáles son las áreas de más alta prioridad para analizar?	Gestión de secretos, validación multiusuario, QA móvil, datos de prueba consistentes y cobertura automatizada.

2.9.2. 9.2 Evaluación objetiva de roles

Rol	¿Qué funcionó?	¿Dónde estuvieron los problemas?	¿Dónde se puede mejorar?	Objetivo de mejoramiento para el Milestone 4
Scrum Master / Liderazgo	Scrum Poker mejoró estimación y compromiso colectivo.	La revisión de riesgos de seguridad no estuvo en la agenda inicial.	Incluir seguridad como ítem fijo de planeación.	Evitar que riesgos de secretos aparezcan al final del sprint.
Product Owner / Planeación	Mantuvo coherencia entre dominio, documentos y relaciones.	No todos los criterios visuales incluyeron responsive.	Agregar escenarios móviles a historias visuales.	Cerrar historias con criterios de UX y privacidad explícitos.
Configuration Manager / Soporte	Identificó la necesidad de .env.example y control de secretos.	La detección llegó por SonarCloud, no por revisión previa.	Automatizar detección temprana de credenciales.	No permitir credenciales hardcodeadas en ramas.
QA Lead / Calidad	Estandarizó reportes de bugs con reproducción.	La cobertura móvil no fue criterio inicial.	Incluir matriz de dispositivos/resoluciones.	Validar componentes de estado en escritorio y móvil.
DevOps Engineer / Proceso de entrega	CI/CD y SonarCloud detectaron problemas reales de seguridad.	Los controles se activaron tarde respecto al flujo de desarrollo.	Agregar hooks locales/pre-commit.	Detectar secretos antes de push.
Ingenieros / Desarrollo	Implementaron persistencia real y validaciones de dominio.	Algunas validaciones requirieron retrabajo por complejidad relacional.	Diseñar contratos de datos antes de rutas y UI.	Validar modelos, rutas y UI de forma integrada.

2.9.3. 9.3 Reporte del ciclo por responsabilidades

Responsabilidad solicitada en la PPT	Respuesta del milestone
Resumen del ciclo	Ciclo de madurez técnica: el producto ganó nube, privacidad y QA formal; el principal aprendizaje fue seguridad preventiva.
Liderazgo: motivación, compromisos, reuniones y apoyo requerido	La motivación aumentó por avances reales de nube y QA; el compromiso se afectó temporalmente por remediación de secretos.
Desarrollo: contenido producido frente a requisitos	El producto cumplió requisitos de persistencia y relaciones centrales, aunque requirió ajustes de seguridad y responsividad.
Planeación: tareas, seguimiento y compromisos	Scrum Poker mejoró la planeación, pero no anticipó suficientemente la complejidad de validaciones y configuración.
Proceso: disciplina de trabajo, medición y seguimiento	El proceso fue más medible por QA y pipeline, aunque todavía faltaban controles locales preventivos.
Calidad: estándares, inspecciones, defectos y PIP	La calidad subió por reportes estandarizados y SonarCloud; los defectos fueron más rastreables.
Soporte: logística, configuración, cambios y riesgos	La configuración de nube funcionó, pero la política de secretos debía estar definida antes.
Reporte de ingenieros: planeado vs. ejecutado y mejora personal	Los ingenieros ejecutaron tareas complejas y aprendieron a medir mejor el trabajo, pero deben incorporar seguridad y responsive desde el inicio.

2.9.4. 9.4 Estrategia de desarrollo y atributos de calidad

Pregunta guía	Respuesta
¿La estrategia de desarrollo funcionó?	Funcionó en migrar a nube y formalizar QA; falló parcialmente por tratar secretos como configuración tardía.
¿Qué otros enfoques hubieran sido más adecuados?	Hubiera sido más adecuado iniciar el sprint con <code>.env.example</code> , variables protegidas y hooks de detección de secretos.
¿Cómo debería cambiarse la estrategia en el futuro?	En futuros ciclos, cualquier feature que toque datos debe iniciar con modelo, seguridad, entorno y pruebas de aislamiento.
¿Cómo se tuvo en cuenta la usabilidad?	Se revisó visualmente el StatusBadge y se identificaron problemas móviles.
¿Cómo se tuvo en cuenta la mantenibilidad?	Mejoró al estabilizar modelos y relaciones de dominio antes del M4.
¿Cómo se tuvo en cuenta la compatibilidad?	La nube redujo dependencia de entornos locales, pero se mantuvo la necesidad de pruebas móviles.
¿Cómo se tuvo en cuenta el desempeño?	MongoDB Atlas habilitó persistencia real; no se midió performance con carga, pero se validó operación funcional.

Pregunta guía	Respuesta
¿Cómo se tuvo en cuenta la seguridad?	Se implementó aislamiento por <code>ownerEmail</code> , pero hubo falla crítica de proceso por credenciales hardcodeadas.
¿Cómo mejorar estos tópicos en el futuro?	Convertir seguridad, responsive y datos de prueba en criterios obligatorios de aceptación y pipeline.

2.9.5. 9.5 Administración de configuración, riesgos y reutilización

Pregunta guía	Respuesta
¿Funcionó la administración de configuración?	Funcionó para conectar a nube, pero falló en prevenir secretos hardcodeados.
¿Funcionó el control de cambios?	Los cambios se controlaron mejor con QA e issues, aunque faltó revisión preventiva de configuración.
¿Cómo mejorarlos?	Plantilla <code>.env.example</code> , secretos fuera del repo, pre-commit hooks y documentación de variables.
¿Fue efectivo el manejo y seguimiento de riesgos?	Mejoró respecto a M2, pero los riesgos de seguridad se trataron tarde.
¿Fue buena la estrategia de reutilización?	La reutilización de modelos y componentes fue más sólida, aunque debe apoyarse con contratos claros.

2.9.6. 9.6 Reflexión del trabajo en grupo

Aspecto de reflexión	Respuesta
Comunicación del grupo	La comunicación mejoró con reportes QA claros, aunque la seguridad exigía discusión temprana.
Calidad del trabajo de los integrantes	La calidad se volvió más objetiva por evidencias y pasos de reproducción.
Planeación de tareas y seguimiento	Scrum Poker aumentó realismo, pero faltó sumar riesgos de seguridad.
Mitigación de riesgos	Se detectaron riesgos por herramientas, no por prevención humana temprana.
Relación con el cliente / stakeholder académico	MongoDB Atlas y aislamiento por usuario elevaron el valor percibido del sistema.
Relación con las tecnologías	El equipo manejó nube, CI/CD y validaciones multiusuario con mayor madurez.

2.9.7. 9.7 Verificación Starfish: respuestas por eje

Eje de la estrella	Pregunta de la PPT que responde	Respuesta concreta del milestone
Comenzar a hacer	¿Qué acciones deberíamos comenzar para lograr el éxito, resolver problemas, mitigar riesgos, mejorar calidad, aprender, usar herramientas, distribuir tiempo y comunicar mejor?	Configurar <code>.env</code> , hooks de secretos, responsive criteria, limpieza de datos y reportes de bugs desde el inicio.
Más de	¿Qué acciones que ya hacemos deberíamos hacer más para mejorar productividad, comunicación, calidad y aprendizaje?	Hacer más peer reviews, pruebas en dispositivos, trazabilidad y controles de datos reales.
Seguir haciendo	¿Qué acciones que ya hacemos bien deberíamos continuar?	Mantener Scrum Poker, QA formal, MongoDB Atlas y reportes reproducibles.
Menos de	¿Qué acciones deberíamos hacer menos porque causan inconvenientes o reducen calidad, comunicación o planeación?	Reducir configuración tardía, dependencia de datos simulados y validaciones sin casos de prueba.
Dejar de hacer	¿Qué acciones deberíamos dejar de hacer porque no son productivas o ya no se necesitan?	Dejar de incluir credenciales hardcodeadas y de tratar seguridad como tarea final.

2.9.8. 9.8 PIP - Plan de Mejora del Proceso

Mejora priorizada	Problema que ataca	Cambio concreto en el proceso	Evidencia esperada
Gestión de secretos desde Día 1	Credenciales hardcodeadas	Crear <code>.env.example</code> , usar variables protegidas y hooks	SonarCloud sin hallazgos de secretos
QA responsive	StatusBadge falló en móvil	Agregar matriz de resoluciones a historias visuales	Evidencias en escritorio y móvil
Datos de prueba limpios	Resultados inconsistentes	Automatizar limpieza y semilla controlada	QA reproducible
Revisión de riesgos en planning	Riesgos detectados tarde	Incluir checklist de seguridad y datos	Riesgos registrados antes de codificar